1. Introduction

We generally use Analytics across almost all our public websites to track unique users. Mostly, we use third-party services for these cases, such as Google Analytics. But have you ever thought about how to track the unique users who visited?

Let's define our goal as calculating the unique visitor count for each day using their IP address for the website `https://vuteem.vercel.app`.

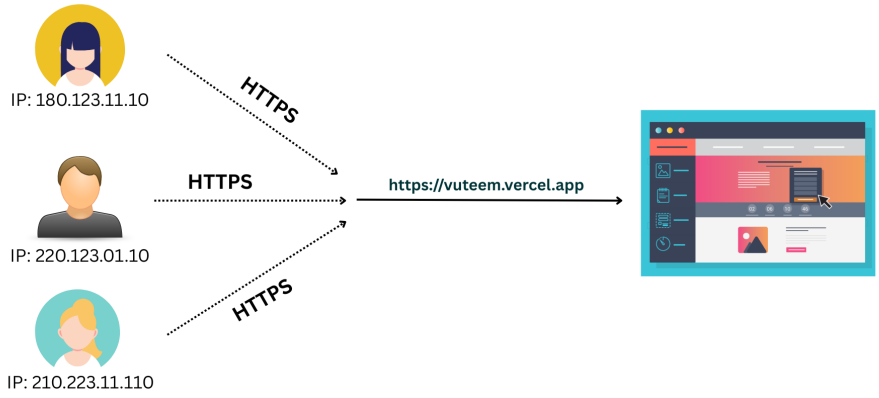

Figure 1: Multiple users with different IP addresses visiting `https://vuteem.vercel.app`

The naive and straightforward approach should be to:

- Retrieve all unique IP addresses from the session DB and get the count.

```
SELECT COUNT(ip) AS unique_users_count FROM session_db;
```

Well, what if we have a billion rows in the `session_db` table? Hence, we are scanning the entire table to count distinct IP addresses for users, which is time-consuming.

What could be a better solution here? Hence, we will learn about a very smart, yet somewhat underrated algorithm called **HyperLogLog** - cardinality estimation algorithm, for estimating the number of unique users without scanning the table in this article. Let's see how the algorithm actually works to understand it better.

2. HyperLogLog Algorithm

HyperLogLog algorithm generally works in two steps,

- Add items into buckets `HLL.add(userIP)` or any identifier
- Estimate the number of unique items from the buckets `HLL.count()`

Now we will explore these two functions in detail.

2.1 HLL.add(userIP)

First, we will calculate a 64-bit binary hash of the user's given IP address. As we don't need cryptographic security here, we can use any lightweight, faster function that distributes the 0/1 binary value more effectively. e.g. `MurmurHash3`, `XXHash64`, etc.

```
const hashedUserIp = hash(userIp)
```

Now, based on the first n binary digits, we determine how many buckets to use to store the user's count info. Choosing this value will tell us how much storage we need for buckets. For choosing the first 10 bits out of 64 bits, we will have $2^{10} = 1024$ buckets, each bucket storing an integer number, which costs 1024×4 bytes (4bytes for 32-bit integer), for a total of 4 KB of storage. So, using only 4 KB of storage, we can estimate the number of unique users who visited our site with just $\mathcal{O}(1024)$ time, which is nearly constant. But choosing 1024 buckets results in a 3.25% error rate when estimating unique user counts. By increasing the bucket size to 2^{16} which increases the first n bits to 16, we will have 65,536 buckets and reduce the error rate to 0.41%, but it will also increase the memory size to $65{,}536 \times 4$ bytes $= 256$ KB, and the order of $\mathcal{O}(65{,}536)$, which is still faster than scanning the full billion-rows `session_db` table.

2.1.1 1st User Visit : HLL.add(180.123.11.10)

Let's say we have:

$$\texttt{hashedUserIp} \leftarrow \texttt{Hash(180.123.11.10)}$$

which gives the following binary:

$$\underbrace{1010110010101101}_{\text{first 16 bits}}\underbrace{0010001010010110111000100101110001010110011010110}_{\text{remaining 48 bits}}$$

Let's convert the first $n = 16$ digits 1010110010101101 into decimal To select the bucket number:

$$(1010110010101101)_2 = (44205)_{10}$$

So, for the user with IP `180.123.11.10`, it will be placed in `bucket[44205]`, where we can have at most 65,536 buckets.

In the next step, we will calculate ρ, the position of the leftmost 1-bit in the remaining 48 bits. Formally:

$$\rho(w) = \text{leading zeros of } w + 1$$

This value tells us how "rare" the observed pattern is—the higher ρ, the rarer the pattern.

`001000101001011011100010010111000101010110011010110`

Here, the remaining 48 values have 2 leading zeros, so $\rho = 2 + 1 = 3$, and we store:

$$\texttt{bucket[44205]} = \max(\textit{initial\_value},\ 3) = 3, \quad \text{where } \textit{initial\_value} = 0$$

2.1.2 2nd User Visit: HLL.add(220.123.01.10)

For the next user, let's assume:

$$\texttt{hashedUserIp} \leftarrow \texttt{Hash(220.123.01.10)}$$

which gives the following binary:

$$\underbrace{0000000000001010}_{\text{first 16 bits}}\underbrace{01101110100101101110001001011100010101110110101 1}_{\text{remaining 48 bits}}$$

Let's convert the first $n = 16$ digits `0000000000001010` into decimal To select the bucket number:

$$(0000000000001010)_2 = (10)_{10}$$

So, for the user with IP `220.123.01.10`, it will be placed in `bucket[10]`.

In the next step, we will calculate the leading zeros of the remaining 48 bits.

`011011101001011011100010010111000101010111011010 11`

Here, the remaining 48 values have 1 leading zero, so $\rho = 1 + 1 = 2$, and we store:

$$\texttt{bucket[10]} = \max(\textit{initial\_value},\ 2) = 2, \quad \text{where } \textit{initial\_value} = 0$$

And for each next user visit, we will call `HLL.add(userIP)`, which will update the value of the `bucket[0]` through `bucket[65535]`.

2.2 HLL.count()

Now we will jump to the next step: the next function, `HLL.count()`. Let's See how it works to estimate the total number of unique users.

Here, the first step is to calculate the combined strength (probability) of the 0/1 hashed pattern. For our 1st case, we have `bucket[10]` $= 2$, which means $\frac{1}{2^2} = 25\%$ probability; for another example, let's say `bucket[50]` $= 4$, the probability is $\frac{1}{2^4} = 6.25\%$. So `bucket[50]` has a rarer pattern than `bucket[10]`. The higher the ρ value stored in a bucket, the rarer the observed pattern.

So for our `bucket[0]`, `bucket[1]`, ..., `bucket[10]`, ..., `bucket[50]`, ..., `bucket[44205]`, ..., `bucket[65535]`, we will calculate total strength:

$$S = \frac{1}{2^{bucket[0]}} + \frac{1}{2^{bucket[1]}} + \cdots + \frac{1}{2^{bucket[n-2]}} + \frac{1}{2^{bucket[n-1]}}$$

which is actually:

$$S = \sum_{i=0}^{m-1} \frac{1}{2^{bucket[i]}} = \sum_{i=0}^{m-1} 2^{-bucket[i]}$$

Where m is the bucket size (in our case, 65,536), and S is the combined strength of the pattern found in the remaining 54 bits, with a leading 0 for all 65,536 buckets.

After finding the strength, we will calculate the actual result using the estimate function:

$$\boxed{\hat{n} = \frac{\alpha(m) \times m^2}{S}}$$

Where $\alpha(m)$ is the bias correction constant. See Flajolet et al. [2007] section 3.2 for the derivation of $\alpha(m)$. For $m \geq 128$, $\alpha(m)$ is approximately $0.7213/(1 + 1.079/m)$.

Now, from this **HLL.count()** function, let's return the estimated value $\hat{\boldsymbol{n}}$ as the count.

3. Conclusion

Many databases, by default, provide this HyperLogLog function. e.g., Redis, Oracle, Microsoft SQL Server. For PostgreSQL, the hll extension could be used. But still, it's worth knowing how this algorithm works under the hood.

References

Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm, 2007. URL https://inria.hal.science/hal-00406166/file/dmAH0110.pdf. HAL preprint.